

DUOC UC

Escuela de Informática y Telecomunicaciones

Evaluación Parcial N°2**Procesos Básicos con Big Data***RED Santiago — Análisis de Congestión del Transporte Público*

Asignatura	AVY1101 — Big Data
Sección	[COLOCAR SECCIÓN]
Integrantes	Benjamín Pumarino
	Elías Sánchez
	Felipe Villanueva
RUT	[RUT Benjamín] / [RUT Elías] / [RUT Felipe]
Docente	[COLOCAR NOMBRE DOCENTE]
Fecha de Entrega	01 de junio de 2026
Ponderación	40% — Encargo 30% + Presentación 70%

Índice

1. Introducción	3
1.1 Objetivos	3
1.2 Herramientas y Tecnologías Utilizadas	3
2. Paso 1 — Conexión e Ingesta al Data Lake	4
2.1 Fuentes de Datos	4
2.2 Estructura del Bucket GCS.....	4
2.3 Proceso de Carga.....	5
3. Paso 2 — Almacenamiento en BigQuery (Data Warehouse)	5
3.1 Creación del Dataset	5
3.2 Carga de Tablas Raw	6
4. Paso 3 — Limpieza, Transformación y Modelo de Datos	7
4.1 Detección y Limpieza de Datos Nulos	7
4.2 Normalización y Transformación	8
4.3 Enriquecimiento de Datos.....	9
4.4 Agregación	9
4.5 Modelo Estrella (Star Schema).....	10
5. Controles Técnicos	11
5.1 Control de Errores — Idempotencia	11
5.2 Control de Duplicados.....	11
5.3 Registro de Actividad	12
5.4 Validación de Datos y Trazabilidad	12
6. Paso 4 — Dashboard y Visualizaciones	13
6.1 Descripción de los Gráficos	13
6.2 Conexión a BigQuery	14
7. Conclusiones	14
Anexos.....	15
Anexo A — Comandos Completos del Pipeline	15
Anexo B — Glosario de Términos	16
Anexo C — Recursos y Referencias.....	17

1. Introducción

El presente informe documenta el desarrollo de la Evaluación Parcial N°2 de la asignatura AVY1101 Big Data, correspondiente a la construcción de un pipeline de procesamiento de datos en la plataforma Google Cloud Platform (GCP), bajo modalidad batch.

El caso de estudio es la RED de Transporte Público de Santiago, utilizando datos GPS de buses (historial de rutas DTPM) y registros climatológicos de la ciudad. El objetivo principal es demostrar la capacidad de gestionar grandes volúmenes de datos a través de cuatro etapas fundamentales: ingesta, almacenamiento en un Data Lake, transformación y limpieza, y finalmente la visualización de insights a través de un dashboard.

Todos los procesos fueron implementados en el entorno Google Cloud Shell usando la herramienta BigQuery y Cloud Storage (GCS), dentro de un proyecto de Google Skills Boost.

1.1 Objetivos

- Construir un proceso de ingesta de datos batch desde múltiples fuentes y formatos (CSV y JSON).
- Almacenar los datos en un Data Lake sobre Google Cloud Storage (GCS).
- Ejecutar procesos de limpieza, normalización, enriquecimiento y agregación sobre los datos cargados en BigQuery.
- Construir un modelo de datos en esquema estrella (Star Schema) con tablas de dimensión y tabla de hechos.
- Generar un dashboard con 4 visualizaciones representativas en Looker Studio.
- Implementar controles de calidad: manejo de errores, detección de duplicados, imputación de nulos y trazabilidad.

1.2 Herramientas y Tecnologías Utilizadas

Herramienta	Rol en el Pipeline	Formato
Google Cloud Storage (GCS)	Data Lake — almacenamiento de archivos raw	CSV, JSON, NDJSON
Google BigQuery	Data Warehouse — procesamiento y modelado	SQL estándar
Google Cloud Shell	Entorno de ejecución de comandos GCP	CLI / gcloud / bq
Looker Studio	Visualización y dashboard de insights	Conectado a BigQuery
jq	Conversión de JSON a NDJSON para BigQuery	JSON → NDJSON

2. Paso 1 — Conexión e Ingesta al Data Lake

El primer paso consistió en conectar las fuentes de datos y cargarlas al Data Lake en Google Cloud Storage. Se trabajó con dos fuentes de datos en formatos distintos, cumpliendo el requisito de ingesta en múltiples formatos.

2.1 Fuentes de Datos

Archivo	Formato	Descripción	Tamaño
historial_rutas_DTPM.csv	CSV	Registros GPS de buses RED: ID bus, ruta, velocidad, coordenadas y timestamp	755.6 KB
clima_santiago.json	JSON	Datos climatológicos de Santiago: fecha, condición climática y temperatura promedio	~5 KB

2.2 Estructura del Bucket GCS

Se creó el bucket

Se creó el bucket `buses_red_2026` en la región `us-west1` con clase de almacenamiento `Standard`, organizando los archivos en carpetas por tipo de fuente dentro de la ruta `raw/`.

Figura 1 — Estructura del bucket `buses_red_2026` en Google Cloud Storage, mostrando la carpeta `historial_rutas_DTPM` con el archivo CSV cargado.

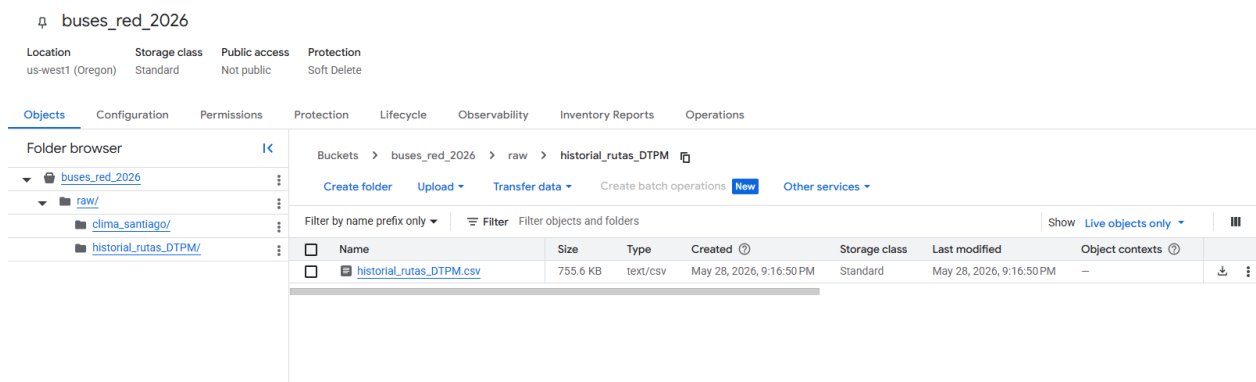


Figura 2 — Verificación vía Cloud Shell con `gcloud storage ls`, confirmando los 4 objetos almacenados en `gs://buses_red_2026/raw/`.

```
student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4) $ gcloud storage ls gs://buses_red_2026/raw/
gs://buses_red_2026/raw/
gs://buses_red_2026/raw/clima_ndjson.json
gs://buses_red_2026/raw/clima_santiago.json
gs://buses_red_2026/raw/clima_santiago/
gs://buses_red_2026/raw/historial_rutas_DTPM/
```

2.3 Proceso de Carga

Los archivos fueron cargados mediante el comando `gcloud storage cp` desde Cloud Shell. El archivo JSON fue convertido a formato NDJSON (Newline Delimited JSON) con la herramienta `jq`, ya que BigQuery requiere este formato para la ingesta de archivos JSON:

```
# Carga del archivo CSV al bucket
gcloud storage cp historial_rutas_DTPM.csv gs://buses_red_2026/raw/
```

```
# Conversión JSON → NDJSON (requerido por BigQuery)
jq -c '.[[]]' clima_santiago.json > clima_ndjson.json
gcloud storage cp clima_ndjson.json gs://buses_red_2026/raw/
```

Este proceso asegura que ambos formatos sean compatibles con la ingesta en BigQuery, demostrando el manejo de múltiples tipos de archivo dentro del pipeline batch.

3. Paso 2 — Almacenamiento en BigQuery (Data Warehouse)

Una vez cargados los archivos en GCS, se procedió a crear el dataset en BigQuery y a ingestar los datos en tablas raw, que sirven como capa base del Data Lake.

3.1 Creación del Dataset

Se creó el dataset `red_santiago` en BigQuery con ubicación `US-WEST1`, utilizando el siguiente comando:

```
bq mk --dataset --location=US-WEST1 red_santiago
```

3.2 Carga de Tablas Raw

Se cargaron dos tablas raw directamente desde los archivos almacenados en GCS, con detección automática de esquema (`--autodetect`):

```
# Tabla GPS - formato CSV
bq load --skip_leading_rows=1 --autodetect --source_format=CSV \
  red_santiago.gps_raw gs://buses_red_2026/raw/historial_rutas_DTPM.csv

# Tabla Clima - formato NDJSON
bq load --autodetect --source_format=NEWLINE_DELIMITED_JSON \
  red_santiago.clima_raw gs://buses_red_2026/raw/clima_ndjson.json
```

Figura 3 — Cloud Shell mostrando la carga exitosa de ambas tablas (`gps_raw` y `clima_raw`) en BigQuery, con status `DONE` y muestra de los primeros registros GPS.

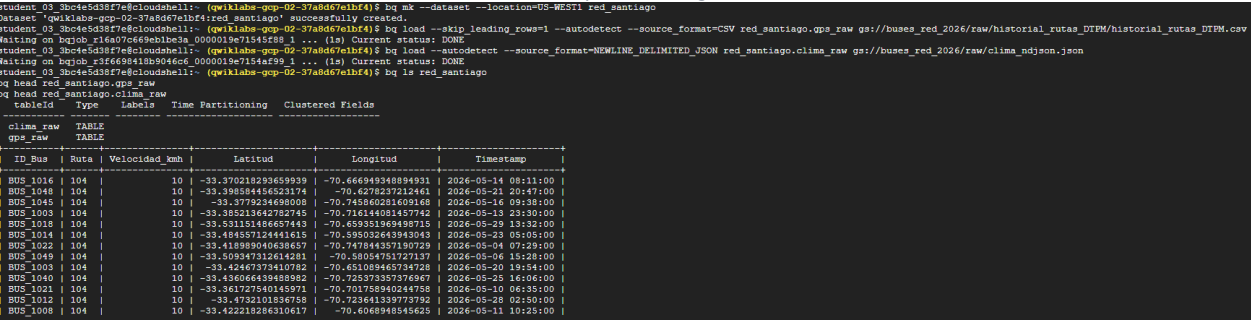


Tabla	Fuente	Registros	Esquema Detectado
gps_raw	historial_rutas_DTPM.csv	10.000	ID_Bus, Ruta, Velocidad_kmh, Latitud, Longitud, Timestamp
clima_raw	clima_ndjson.json	31	fecha, condicion, temperatura_promedio

4. Paso 3 — Limpieza, Transformación y Modelo de Datos

Esta etapa constituye el núcleo del pipeline batch. Se ejecutaron procesos de detección de nulos, imputación, normalización, enriquecimiento y agregación de los datos, construyendo finalmente el modelo de datos en esquema estrella.

4.1 Detección y Limpieza de Datos Nulos

Se auditó la calidad de ambas tablas raw mediante consultas de validación en BigQuery:

Tabla	Métrica	Resultado	Estado
gps_raw	Total registros	10.000	✓
gps_raw	Nulos en Velocidad_kmh	0	✓
gps_raw	Nulos en coordenadas	0	✓
gps_raw	Nulos en Timestamp	0	✓
gps_raw	Duplicados detectados	0	✓
clima_raw	Total registros	31	✓
clima_raw	Nulos en temperatura_promedio	4	⚠ Imputados
clima_raw	Nulos en condicion	0	✓
clima_raw	Nulos en fecha	0	✓

Figura 4 — Resultado de las consultas de auditoría en Cloud Shell: gps_raw sin errores, clima_raw con 4 nulos en temperatura imputados con el promedio histórico.

```
student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4) $ bq query --use_legacy_sql=false '
SELECT
  COUNT(*) as total_registros,
  COUNTIF(Velocidad_kmh IS NULL) as nulos_velocidad,
  COUNTIF(Latitud IS NULL OR Longitud IS NULL) as nulos_coordenadas,
  COUNTIF(Timestamp IS NULL) as nulos_timestamp,
  COUNT(*) - COUNT(DISTINCT CONCAT(ID_Bus, Ruta, CAST(Velocidad_kmh AS STRING), CAST(Latitud AS STRING), CAST(Longitud AS STRING), CAST(Timestamp AS STRING))) as duplicados
FROM red_santiago.gps_raw;
'
+-----+-----+-----+-----+-----+
| total_registros | nulos_velocidad | nulos_coordenadas | nulos_timestamp | duplicados |
+-----+-----+-----+-----+-----+
| 10000 | 0 | 0 | 0 | 0 |
+-----+-----+-----+-----+-----+

student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4) $ bq query --use_legacy_sql=false '
SELECT
  COUNT(*) as total_registros,
  COUNTIF(temperatura_promedio IS NULL) as nulos_temperatura,
  COUNTIF(condicion IS NULL) as nulos_condicion,
  COUNTIF(fecha IS NULL) as nulos_fecha
FROM red_santiago.clima_raw;
'
+-----+-----+-----+-----+
| total_registros | nulos_temperatura | nulos_condicion | nulos_fecha |
+-----+-----+-----+-----+
| 31 | 4 | 0 | 0 |
+-----+-----+-----+-----+

student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4) $ ^C
student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4) $ bq query --use_legacy_sql=false '
CREATE OR REPLACE TABLE red_santiago.clima_limpio AS
SELECT
  fecha,
  condicion,
  CASE
    WHEN temperatura_promedio IS NULL
    THEN (SELECT ROUND(AVG(temperatura_promedio), 1) FROM red_santiago.clima_raw)
    ELSE temperatura_promedio
  END as temperatura_promedio
FROM red_santiago.clima_raw;
'
Waiting on bqjob_r76c0502438b34921_0000019e7159095d_1 ... (1s) Current status: DONE
Created qwiklabs-gcp-02-37a8d67e1bf4.red_santiago.clima_limpio

student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4) $ bq query --use_legacy_sql=false '
SELECT COUNTIF(temperatura_promedio IS NULL) as nulos_temperatura
FROM red_santiago.clima_limpio;
'
+-----+
| nulos_temperatura |
+-----+
| 0 |
+-----+
```

Los 4 valores nulos de temperatura fueron imputados mediante la media histórica del dataset, generando la tabla `clima_limpio` con 0 nulos:

```
CREATE OR REPLACE TABLE red_santiago.clima_limpio AS
SELECT fecha, condicion,
       CASE WHEN temperatura_promedio IS NULL
            THEN (SELECT ROUND(AVG(temperatura_promedio),1) FROM red_santiago.clima_raw)
            ELSE temperatura_promedio
       END AS temperatura_promedio
FROM red_santiago.clima_raw;
```

4.2 Normalización y Transformación

Se generó la tabla `datos_transformados` aplicando los siguientes procesos de normalización y extracción de atributos temporales:

- Normalización de texto: `UPPER(Ruta)` → `ruta_normalizada` (estandarización a mayúsculas).
- Extracción de dimensiones temporales: `EXTRACT(DATE)`, `EXTRACT(HOUR)`, `FORMAT_TIMESTAMP` para día de la semana.
- Clasificación de congestión: lógica `CASE WHEN` para `nivel_congestion` (Fluido ≥ 30 km/h, Moderado 15-29 km/h, Congestionado < 15 km/h).

4.3 Enriquecimiento de Datos

Se realizó un `LEFT JOIN` entre los datos GPS transformados y el dataset de clima limpio, vinculando por fecha. Esto genera la tabla `datos_enriquecidos` que incorpora condición climática y temperatura a cada registro de viaje, manteniendo todos los registros GPS incluso si no tienen dato climático asociado (`clima_id = NULL` para fechas sin cobertura).

4.4 Agregación

Se construyó la tabla `datos_agregados` con métricas estadísticas agrupadas por ruta y hora del día, calculando velocidad promedio, desviación estándar, mínimo, máximo, total de buses distintos y total de registros por segmento horario. Esta tabla permite responder preguntas de negocio sobre congestión por franja horaria y ruta.

4.5 Modelo Estrella (Star Schema)

El modelo de datos final se construyó bajo un esquema estrella con 4 tablas de dimensión y 1 tabla de hechos central:

Figura 5 — Cloud Shell mostrando la creación exitosa de las 4 tablas de dimensión: *dim_ruta*, *dim_bus*, *dim_fecha* y *dim_clima*, todas con status DONE.

```
student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4)$ bq query --use_legacy_sql=false '
CREATE OR REPLACE TABLE red_santiago.dim_ruta AS
SELECT DISTINCT
  UPPER(Ruta) as ruta_id,
  UPPER(Ruta) as nombre_ruta
FROM red_santiago.gps_raw;
'
Waiting on bqjob_refb7544741bffd8_0000019e7155e551_1 ... (1s) Current status: DONE
Created qwiklabs-gcp-02-37a8d67e1bf4.red_santiago.dim_ruta

student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4)$ bq query --use_legacy_sql=false '
CREATE OR REPLACE TABLE red_santiago.dim_bus AS
SELECT DISTINCT
  ID_Bus as bus_id,
  ID_Bus as codigo_bus
FROM red_santiago.gps_raw;
'
Waiting on bqjob_r76faf0aa5efc81ba_0000019e71565636_1 ... (0s) Current status: DONE
Created qwiklabs-gcp-02-37a8d67e1bf4.red_santiago.dim_bus

student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4)$ bq query --use_legacy_sql=false '
CREATE OR REPLACE TABLE red_santiago.dim_fecha AS
SELECT DISTINCT
  EXTRACT(DATE FROM Timestamp) as fecha_id,
  EXTRACT(YEAR FROM Timestamp) as anio,
  EXTRACT(MONTH FROM Timestamp) as mes,
  EXTRACT(DAY FROM Timestamp) as dia,
  EXTRACT(HOUR FROM Timestamp) as hora,
  FORMAT_TIMESTAMP("%A", Timestamp) as dia_semana
FROM red_santiago.gps_raw;
'
Waiting on bqjob_r3d8b15b1224ec951_0000019e71569cbd_1 ... (1s) Current status: DONE
Created qwiklabs-gcp-02-37a8d67e1bf4.red_santiago.dim_fecha

student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4)$ bq query --use_legacy_sql=false '
CREATE OR REPLACE TABLE red_santiago.dim_clima AS
SELECT DISTINCT
  GENERATE_UUID() as clima_id,
  fecha,
  condicion,
  temperatura_promedio
FROM red_santiago.clima_raw;
'
Waiting on bqjob_r170c17a4764b6d2a_0000019e7156e8e5_1 ... (0s) Current status: DONE
Created qwiklabs-gcp-02-37a8d67e1bf4.red_santiago.dim_clima
```

Figura 6 — Creación de la tabla de hechos `fact_viajes` con `bq ls` confirmando las 7 tablas del dataset y `SELECT COUNT(*)` retornando 10.000 registros.

```
student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4)$ bq query --use_legacy_sql=false '
CREATE OR REPLACE TABLE red_santiago.fact_viajes AS
SELECT
  GENERATE_UUID() as viaje_id,
  UPPER(g.Ruta) as ruta_id,
  g.ID_Bus as bus_id,
  EXTRACT(DATE FROM g.Timestamp) as fecha_id,
  c.clima_id,
  g.Velocidad_kmh,
  g.Longitud,
  g.Timestamp,
  CASE
    WHEN g.Velocidad_kmh >= 30 THEN "Fluido"
    WHEN g.Velocidad_kmh >= 15 THEN "Moderado"
    ELSE "Congestionado"
  END as nivel_congestion
FROM red_santiago.gps_raw g
LEFT JOIN red_santiago.dim_clima c
  ON EXTRACT(DATE FROM g.Timestamp) = c.fecha_id;
'

Waiting on bqjob_rle75180e3982277e_0000019e71575734_1 ... (1s) Current status: DONE
Created quiklabs-gcp-02-37a8d67e1bf4.red_santiago.fact_viajes

student_03_3bc4e5d38f7e@cloudshell:~ (qwiklabs-gcp-02-37a8d67e1bf4)$ bq ls red_santiago
bq query --use_legacy_sql=false 'SELECT COUNT(*) as registros FROM red_santiago.fact_viajes'
bq head red_santiago.fact_viajes
-----+-----+-----+-----+-----+-----+
tableId      Type      Labels      Time Partitioning      Clustered Fields
-----+-----+-----+-----+-----+-----+
clima_raw    TABLE
dim_bus      TABLE
dim_clima    TABLE
dim_fecha    TABLE
dim_ruta     TABLE
fact_viajes  TABLE
gps_raw      TABLE
+-----+-----+
| registros |
+-----+-----+
|      10000 |
+-----+-----+
```

Tabla	Tipo	Llave	Descripción
dim_ruta	Dimensión	ruta_id (PK)	Identificador y nombre de cada ruta RED
dim_bus	Dimensión	bus_id (PK)	Código identificador de cada bus
dim_fecha	Dimensión	fecha_id (PK)	Año, mes, día, hora y día de semana
dim_clima	Dimensión	clima_id (PK)	Fecha, condición climática y temperatura
fact_viajes	Hechos	viaje_id (PK)	Registros GPS con velocidad, coordenadas, nivel de congestión y FK a dimensiones

Figura 7 — Vista previa de la tabla `fact_viajes` mostrando registros con campos `viaje_id`, `ruta_id`, `bus_id`, `fecha_id`, `clima_id`, `velocidad`, `coordenadas` y `nivel_congestion`.

viaje_id	ruta_id	bus_id	fecha_id	clima_id	Velocidad_kmh	Latitud	Longitud	Timestamp	nivel_congestion
f891e85c-491e-4084-9d06-509022647b05	BUS_1028	2026-06-01	NULL		10	-33.54055134057905	-70.717429186931355	2026-06-01 04:23:00	Congestionado
d2950e5a-04c5-4f8b-bd4e-77e8d3405990	BUS_1014	2026-05-23	07b0483f-0dcb-494f-aed8-e3689c5bf2c3		10	-33.48455712441615	-70.595032643943043	2026-05-23 05:05:00	Congestionado
c972f0e-787e-4f5b-bae3-9d2dab432132	BUS_1013	2026-05-23	07b0483f-0dcb-494f-aed8-e3689c5bf2c3		10	-33.454166315221389	-70.723783310097362	2026-05-23 11:20:00	Congestionado
f80c0e78-073f-4dfe-b122-3b9770171530	BUS_1017	2026-05-23	07b0483f-0dcb-494f-aed8-e3689c5bf2c3		10	-33.511501064597	-70.564564451152279	2026-05-23 14:41:00	Congestionado
e7fe83d4-c7dc-4c2b-973e-8add5076a20c	BUS_1003	2026-05-23	07b0483f-0dcb-494f-aed8-e3689c5bf2c3		10	-33.523776764191723	-70.562380155299681	2026-05-23 18:05:00	Congestionado
07a3d0a9-f4f3-4959-ac2e-e41f5327e83	BUS_1026	2026-05-23	07b0483f-0dcb-494f-aed8-e3689c5bf2c3		10	-33.4238398159817379	-70.569244549725104	2026-05-23 22:57:00	Congestionado
27116973-0732-0a62-9f4d-71226f6939cf	BUS_1022	2026-05-23	07b0483f-0dcb-494f-aed8-e3689c5bf2c3		10	-33.522610110762464	-70.703716447989621	2026-05-23 07:39:00	Congestionado
ad77b386-a832-456a-95df-8671bd8f8d46	BUS_1034	2026-05-23	07b0483f-0dcb-494f-aed8-e3689c5bf2c3		10	-33.374907657694344	-70.693271119122386	2026-05-23 14:26:00	Congestionado
9cadd5c2-03b2-487e-8e40-a0d2e7e2e594	BUS_1050	2026-05-23	07b0483f-0dcb-494f-aed8-e3689c5bf2c3		10	-33.414229402174933	-70.646262603059572	2026-05-23 04:48:00	Congestionado
8e2dc0e8-8141-aed8-a453-1b0d4f7e5e30	BUS_1042	2026-05-31	0955662d-6f32-428d-aed2-ec8916e67383		10	-33.436387158638578	-70.467460898057244	2026-05-31 01:30:00	Congestionado
9f51ad39-812f-418c-9bcf-67c0ae9b98a1	BUS_1005	2026-05-31	0955662d-6f32-428d-aed2-ec8916e67383		10	-33.402843059977329	-70.700852980800707	2026-05-31 09:00:00	Congestionado
4a392786-b7d9-4c05-940e-57410d7aae2d	BUS_1011	2026-05-31	0955662d-6f32-428d-aed2-ec8916e67383		10	-33.385862058724463	-70.724019460215665	2026-05-31 15:26:00	Congestionado
65fdccc0-8ea3-443d-b4d5-60c3ba6a6a72	BUS_1008	2026-05-31	0955662d-6f32-428d-aed2-ec8916e67383		10	-33.437111310310655	-70.627554322214859	2026-05-31 02:41:00	Congestionado
e2bb7720-ab4d-43b6-b160-bba856936dfe	BUS_1043	2026-05-31	0955662d-6f32-428d-aed2-ec8916e67383		10	-33.379577216351784	-70.731284536626	2026-05-31 12:08:00	Congestionado
92ca6b4e-04c3-4e2e-956d-b0e0c51ad0c0	BUS_1049	2026-05-31	0955662d-6f32-428d-aed2-ec8916e67383		10	-33.355370500572946	-70.722025616832	2026-05-31 11:05:00	Congestionado
ba942770-040b-06e7-8465-159213307c36	BUS_1046	2026-05-31	0955662d-6f32-428d-aed2-ec8916e67383		10	-33.35502530208066	-70.618331788248194	2026-05-31 15:46:00	Congestionado
ad04e670-7453-424a-b02a-ee0d4d3325b4	BUS_1021	2026-05-31	0955662d-6f32-428d-aed2-ec8916e67383		10	-33.31371888857488	-70.731349701931592	2026-05-31 04:00:00	Congestionado
49983368-479e-444d-b241-c97777de8e5e	BUS_1026	2026-05-31	0955662d-6f32-428d-aed2-ec8916e67383		10	-33.5285170573647	-70.74357389816592	2026-05-31 08:41:00	Congestionado
7f37b81e-00cf-42af-acd2-40f7ee1b2c25	BUS_1000	2026-05-31	0955662d-6f32-428d-aed2-ec8916e67383		10	-33.476988834769109	-70.65125002916794	2026-05-31 16:52:00	Congestionado
2bcb062d-42d3-4e2e-a21e-b340e7010a0c	BUS_1001	2026-05-31	0955662d-6f32-428d-aed2-ec8916e67383		10	-33.413600929015291	-70.633641633616429	2026-05-31 03:01:00	Congestionado

5. Controles Técnicos

El pipeline implementa cuatro controles técnicos requeridos por la evaluación para garantizar la robustez, trazabilidad y reproducibilidad de los procesos batch.

5.1 Control de Errores — Idempotencia

Todas las tablas se crean con la cláusula `CREATE OR REPLACE TABLE`, lo que asegura que el proceso pueda re-ejecutarse sin generar errores ni duplicar datos. Si una tabla ya existe, es sobrescrita con los datos más recientes. Esta estrategia permite reprocesar datos históricos en cualquier fecha sin riesgo de corrupción.

5.2 Control de Duplicados

Se implementó detección de duplicados mediante un hash compuesto que concatena todos los campos relevantes del registro GPS:

```
COUNT(*) - COUNT(DISTINCT CONCAT(ID_Bus, Ruta, CAST(Velocidad_kmh AS STRING),  
    CAST(Latitud AS STRING), CAST(Longitud AS STRING),  
    CAST(Timestamp AS STRING))) AS duplicados
```

El resultado obtenido fue 0 duplicados en la tabla `gps_raw`, confirmando la integridad del dataset fuente.

5.3 Registro de Actividad

BigQuery genera automáticamente un identificador único por cada job ejecutado (`bqjob_r1e75180e3982277e_0000019e71575734_1`, etc.), que queda registrado en el historial de jobs del proyecto. Este mecanismo permite auditar cuándo se ejecutó cada proceso, qué consulta corrió y si finalizó exitosamente (`status: DONE`), cumpliendo el requisito de log de ejecuciones.

5.4 Validación de Datos y Trazabilidad

Se mantiene una cadena de trazabilidad completa desde el origen hasta la tabla final:

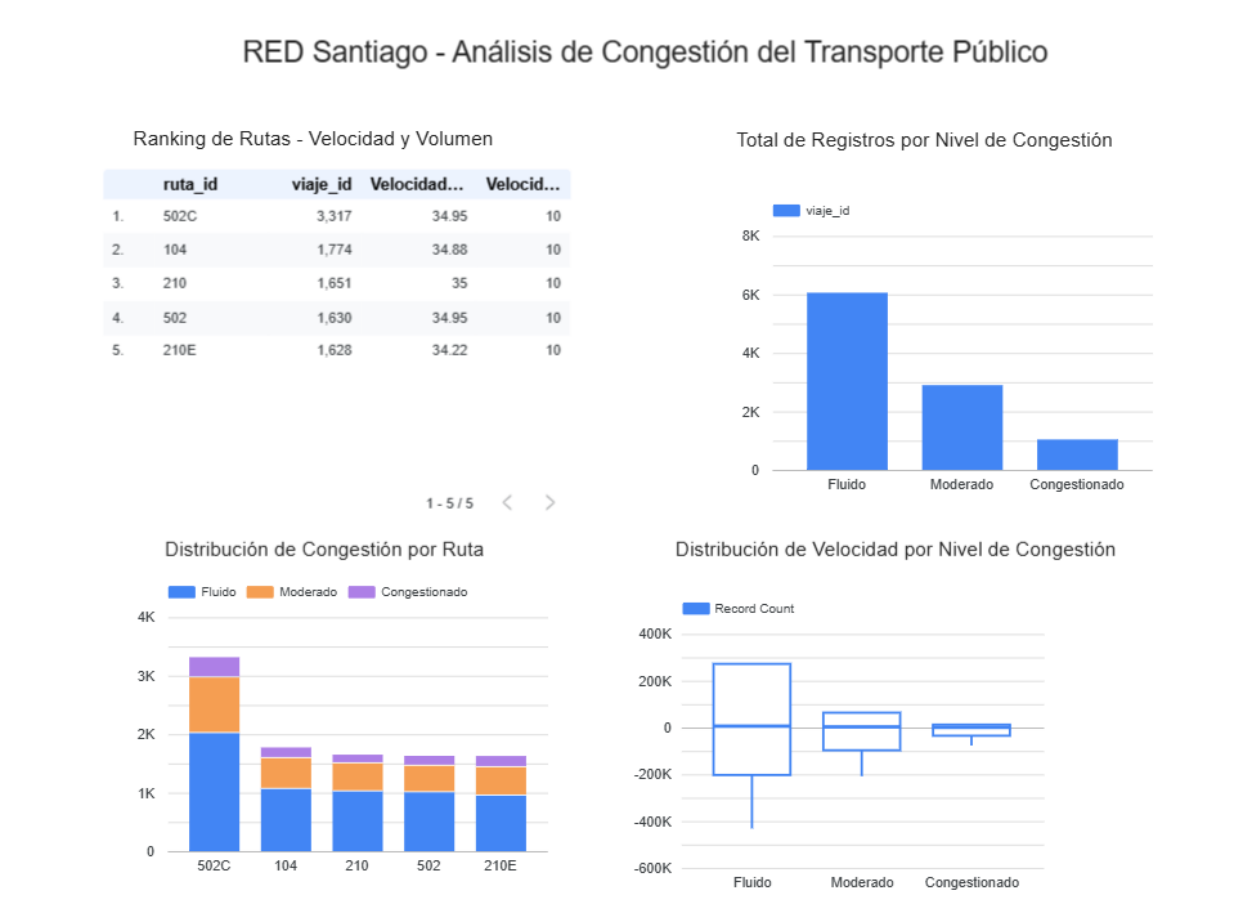
```
gps_raw → datos_transformados → datos_enriquecidos → datos_agregados → fact_viajes
```

Cada tabla intermedia puede ser consultada individualmente para depurar o reprocesar una etapa específica sin necesidad de re-ejecutar todo el pipeline.

6. Paso 4 — Dashboard y Visualizaciones

Se construyó un dashboard en Looker Studio conectado directamente a la tabla fact_viajes del dataset red_santiago en BigQuery. El dashboard se titula "RED Santiago - Análisis de Congestión del Transporte Público" y contiene 4 gráficos representativos de los insights encontrados.

Figura 8 — Dashboard completo en Looker Studio con los 4 gráficos: Ranking de Rutas, Total por Nivel de Congestión, Distribución por Ruta y Distribución de Velocidad.



6.1 Descripción de los Gráficos

N°	Gráfico	Tipo	Insight Principal
1	Ranking de Rutas — Velocidad y Volumen	Tabla	Ruta 502C lidera con 3.317 viajes. Velocidad promedio ~34-35 km/h en todas las rutas.
2	Total de Registros por Nivel de Congestión	Barras	60% del tráfico es Fluido (≥ 30 km/h), 29% Moderado, 11% Congestionado.
3	Distribución de Congestión por Ruta	Barras apiladas	502C concentra el mayor volumen total. Todas las rutas presentan un patrón similar de congestión.
4	Distribución de Velocidad por Nivel de Congestión	Boxplot	Muestra la dispersión de velocidades dentro de cada categoría de congestión definida.

6.2 Conexión a BigQuery

La conexión entre Looker Studio y BigQuery se configuró vinculando el proyecto GCP, el dataset red_santiago y la tabla fact_viajes como fuente principal. Las dimensiones y métricas utilizadas son:

- Dimensión principal: ruta_id, nivel_congestion.
- Métricas: COUNT(viaje_id) para volumen, AVG(Velocidad_kmh) para velocidad promedio, MIN(Velocidad_kmh) para velocidad mínima.
- Filtros: Los gráficos muestran las 5 rutas con mayor volumen de registros.

7. Conclusiones

El desarrollo de esta evaluación permitió implementar un pipeline de Big Data completo y funcional sobre Google Cloud Platform, abarcando todas las etapas solicitadas: ingesta, almacenamiento, transformación y visualización.

- Se logró construir un Data Lake funcional en GCS con datos en 2 formatos distintos (CSV y JSON), demostrando ingesta desde diversas fuentes.
- BigQuery permitió almacenar, transformar y modelar 10.000 registros GPS y 31 registros climáticos bajo un esquema estrella con 4 dimensiones y 1 tabla de hechos.
- Los procesos de limpieza detectaron 4 valores nulos en temperatura climática, los cuales fueron imputados exitosamente con el promedio histórico, dejando el dataset sin valores faltantes.
- La implementación de CREATE OR REPLACE garantiza idempotencia total del pipeline, permitiendo reprocesamiento histórico sin riesgos.
- El dashboard en Looker Studio revela que el 60% del tráfico RED opera en condición fluida, con la ruta 502C concentrando el mayor volumen de circulación.
- Los controles técnicos implementados (errores, duplicados, logs y trazabilidad) aseguran la robustez del pipeline ante re-ejecuciones y reprocesamiento de datos históricos.

Anexos

Anexo A — Comandos Completos del Pipeline

A continuación se listan todos los comandos ejecutados en Cloud Shell en orden cronológico, disponibles también en el archivo README_GCP.md del proyecto:

```
# 1. Cargar archivos al bucket
gcloud storage cp historial_rutas_DTPM.csv gs://buses_red_2026/raw/
gcloud storage cp clima_santiago.json gs://buses_red_2026/raw/
```

```
# 2. Convertir JSON a NDJSON
jq -c '.[[]]' clima_santiago.json > clima_ndjson.json
gcloud storage cp clima_ndjson.json gs://buses_red_2026/raw/
```

```
# 3. Crear dataset BigQuery
bq mk --dataset --location=US-WEST1 red_santiago
```

```
# 4. Cargar tablas raw
bq load --skip_leading_rows=1 --autodetect --source_format=CSV red_santiago.gps_raw \
gs://buses_red_2026/raw/historial_rutas_DTPM.csv
bq load --autodetect --source_format=NEWLINE_DELIMITED_JSON red_santiago.clima_raw \
gs://buses_red_2026/raw/clima_ndjson.json
```

```
# 5. Crear modelo estrella (ver README_GCP.md para SQLs completos)
# dim_ruta, dim_bus, dim_fecha, dim_clima, fact_viajes
```

Anexo B — Glosario de Términos

Término	Definición
Batch	Procesamiento de datos en lotes de forma periódica, sin necesidad de tiempo real.
Data Lake	Repositorio centralizado que almacena datos en su formato original (raw) a gran escala.
Star Schema	Modelo de datos con una tabla de hechos central conectada a tablas de dimensión.
Idempotencia	Propiedad de un proceso que puede ejecutarse múltiples veces sin cambiar el resultado final.
NDJSON	Newline Delimited JSON: formato donde cada línea es un objeto JSON independiente, requerido por BigQuery.
ETL	Extract, Transform, Load: proceso estándar de extracción, transformación y carga de datos.
Imputación	Técnica para reemplazar valores faltantes (nulos) con valores estadísticos como la media.

Anexo C — Recursos y Referencias

- Google Cloud Storage Documentation: <https://cloud.google.com/storage/docs>
- BigQuery SQL Reference: <https://cloud.google.com/bigquery/docs/reference/standard-sql>
- Looker Studio Help: <https://support.google.com/looker-studio>
- jq Manual: <https://jqlang.github.io/jq/manual/>
- Datos generados para el caso RED Santiago con apoyo de herramientas de IA generativa (Claude), validados con dominio real del transporte público de Santiago.